

LAB 7: RECURRENT NEURAL NETWORK (RNN)

OBJECTIVES: Predict sequences using RNN.

THEORY:

Recurrent Neural Networks (RNNs) are a class of neural networks tailored for processing sequential data. Unlike standard architectures that assume independence between inputs, RNNs utilize feedback loops to maintain a persistent hidden state. This state acts as a temporal memory, allowing the network to capture dependencies across different time steps. The output at time step 't' is a function of both the current input and the previous hidden state. This architecture is particularly effective for time-series forecasting, where the sequence's history is critical for predicting future values.

TOOLS: TensorFlow/Keras, NumPy.

DATASET: Synthetic Sine Wave: A time-series dataset generated using the sine function. It consists of sequential values where the objective is to predict the value at time $t+1$ given a window of previous values from $t-n$ to t .

ALGORITHM:

1. Start.
2. Generate a synthetic sine wave sequence and normalize the data to the range $[-1, 1]$.
3. Transform the continuous sequence into a supervised learning format using a sliding window of size W .
4. Reshape the input data into a 3D tensor: $[\text{Batch_Size}, \text{Time_Steps}, \text{Features}]$.
5. Initialize an RNN architecture with a SimpleRNN layer to maintain the hidden state across the sequence.
6. Map the final hidden state to a single output neuron using a Dense layer.
7. Compile the model using the Mean Squared Error (MSE) loss function and the Adam optimizer.
8. Update weights using Backpropagation Through Time (BPTT) over several epochs.
9. Generate predictions for the test sequence.
10. Plot the actual sine wave versus the predicted sequence using distinct line styles for black-and-white distinguishability.
11. End.

SOURCE CODE:

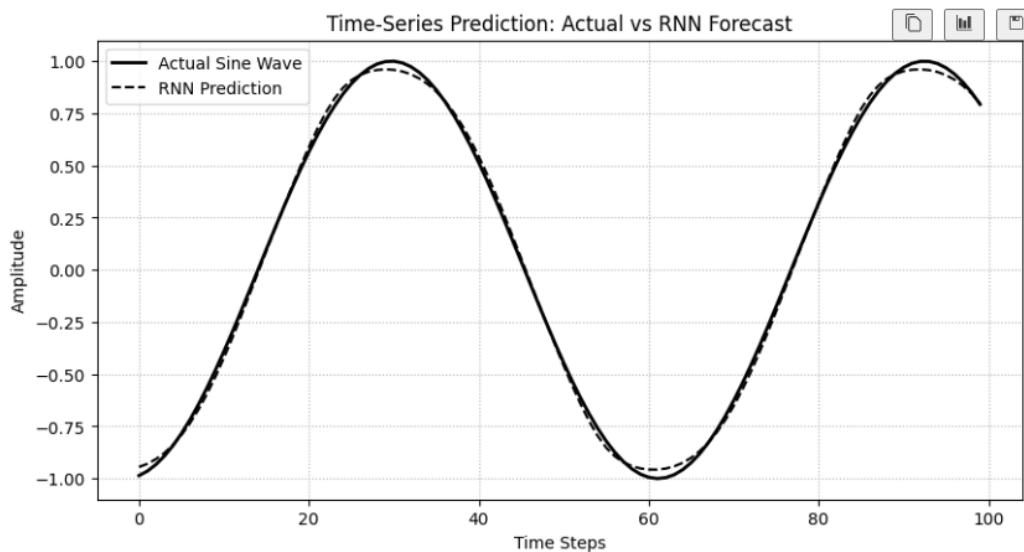
```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense
t = np.linspace(0, 100, 1000)
data = np.sin(t)
# Prepare sliding window dataset (W = 10)
def create_sequences(data, window):
    X, y = [], []
    for i in range(len(data) - window):
        X.append(data[i:i+window])
        y.append(data[i+window])
    return np.array(X), np.array(y)
window_size = 10
X, y = create_sequences(data, window_size)
# Reshape for RNN: [Samples, Time_Steps, Features]
X = X.reshape((X.shape[0], X.shape[1], 1))
# Split into train and test
split = int(0.8 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]
model = Sequential([
    SimpleRNN(32, activation='tanh', input_shape=(window_size, 1)),
    Dense(1)])
model.compile(optimizer='adam', loss='mse')
print("Training RNN on sequence data...")
model.fit(X_train, y_train, epochs=10, batch_size=32, verbose=1)
y_pred = model.predict(X_test)

plt.figure(figsize=(10, 5))
# Using different line styles and widths for B&W printing
plt.plot(y_test[:100], color='black', linewidth=2, label='Actual Sine Wave')
plt.plot(y_pred[:100], color='black', linestyle='--', linewidth=1.5, label='RNN Prediction')

plt.title("Time-Series Prediction: Actual vs RNN Forecast")
plt.xlabel("Time Steps")
plt.ylabel("Amplitude")
plt.legend()
plt.grid(True, linestyle=':')
plt.show()
```

OUTPUTS:

```
c:\Users\N I T R O\AppData\Local\Programs\Python\Python313\Lib\site-packag
if not hasattr(np, "object"):
Training RNN on sequence data...
Epoch 1/10
c:\Users\N I T R O\AppData\Local\Programs\Python\Python313\Lib\site-packag
super().__init__(**kwargs)
25/25 ————— 1s 6ms/step - loss: 0.2397
Epoch 2/10
25/25 ————— 0s 5ms/step - loss: 0.0133
Epoch 3/10
25/25 ————— 0s 6ms/step - loss: 0.0068
Epoch 4/10
25/25 ————— 0s 3ms/step - loss: 0.0044
Epoch 5/10
25/25 ————— 0s 4ms/step - loss: 0.0032
Epoch 6/10
25/25 ————— 0s 3ms/step - loss: 0.0023
Epoch 7/10
25/25 ————— 0s 3ms/step - loss: 0.0020
Epoch 8/10
25/25 ————— 0s 5ms/step - loss: 0.0014
Epoch 9/10
25/25 ————— 0s 4ms/step - loss: 0.0011
Epoch 10/10
25/25 ————— 0s 3ms/step - loss: 7.7849e-04
7/7 ————— 0s 28ms/step
```



CONCLUSION:

The Recurrent Neural Network successfully learned the temporal patterns of the synthetic sine wave. By utilizing feedback connections and maintaining a hidden state, the model effectively predicted the next value in the sequence based on the previous ten time steps. The minimal Mean Squared Error and the visual overlap in the forecast plot demonstrate that RNNs are highly capable of capturing cyclic dependencies in time-series data. The resulting model provides a robust framework for sequential pattern recognition.